Introduction to EKS

Understanding Kubernetes for Orchestration

Introduction

Amazon Elastic Kubernetes Service (EKS) is a managed Kubernetes service that allows users to deploy, scale, and manage Kubernetes applications on AWS **without maintaining Kubernetes infrastructure**.

With EKS, AWS handles the **control plane**, while users manage the **worker nodes** where applications run.

This guide will cover:

- **What is Amazon EKS?**
- **Why use EKS over self-managed Kubernetes?**
- **Key components of EKS**
- **How to set up an EKS cluster**
- **Common issues and FAQs**

What is Amazon EKS?

Amazon EKS is a **fully managed Kubernetes service** that:

- **Automates Kubernetes setup** on AWS.
- **Handles scaling and high availability** across multiple availability zones.
- **Integrates with AWS services** like IAM, CloudWatch, and Load Balancers.
- **Supports native Kubernetes tools** like kubectl, Helm, and Terraform.

💡 *Think of EKS as a Kubernetes cluster where AWS manages the control plane for you, so you focus only on running applications.*

Why Use EKS?

Feature	Benefit
Fully Managed	AWS manages Kubernetes updates, patches, and availability.
High Availability	EKS automatically distributes workloads across AWS regions.
Security	Integrates with AWS IAM, enabling secure access control.
Scalability	Supports auto-scaling for Pods and worker nodes.
Integration	Works with AWS services like VPC, ALB, and CloudWatch.

💡 *EKS is ideal for developers who want a hassle-free Kubernetes setup with AWS-level security and scalability.*

Key Components of EKS

1. **EKS Control Plane** – AWS manages the **API server, scheduler, and etcd database**.
2. **Worker Nodes** – EC2 instances (or Fargate) that run Kubernetes workloads.
3. **VPC (Virtual Private Cloud)** – Provides network isolation for EKS clusters.
4. **IAM (Identity and Access Management)** – Manages authentication for Kubernetes users and applications.
5. **Load Balancers** – Allow external access to Kubernetes applications.

💡 *EKS provides a* **secure, scalable, and highly available Kubernetes environment** *on AWS.*

Setting Up an EKS Cluster

Follow these steps to create an EKS cluster.

Step 1: Install Required Tools

1. **Install AWS CLI**

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
```

2. **Install** kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

3. **Install** eksctl **(EKS CLI Tool)**

```
brew install eksctl
```

Step 2: Create an EKS Cluster

Run the following command to create an EKS cluster with **two worker nodes**:

```
eksctl create cluster --name my-eks-cluster --region ap-south-1 --nodes 2
```

This command:

- Creates a **VPC** and **subnets** for the cluster.
- Deploys a **managed control plane**.

- Adds **two worker nodes** to run applications.

💡 *This is the easiest way to create an EKS cluster.*

Step 3: Verify the Cluster

Check available clusters:

```
eksctl get clusters
```

Configure kubectl to use the new cluster:

```
aws eks update-kubeconfig --region ap-south-1 --name my-eks-cluster
```

Verify connectivity:

```
kubectl get nodes
```

💡 *If you see nodes listed, your cluster is running successfully!*

Deploying an Application on EKS

Step 1: Deploy an Application

Create a simple **Nginx Deployment** on EKS:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
```

```
    app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

Apply the Deployment:

```
kubectl apply -f nginx-deployment.yaml
```

Verify the Deployment:

```
kubectl get deployments
```

💡 *Your application is now running on AWS EKS!*

Accessing an Application on EKS

Create a **LoadBalancer Service** to expose the application:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
```

```
type: LoadBalancer
selector:
  app: nginx
ports:
  - protocol: TCP
    port: 80
    targetPort: 80
```

Apply the Service:

```
kubectl apply -f nginx-service.yaml
```

Retrieve the **external LoadBalancer URL**:

```
kubectl get services
```

💡 *This makes the application publicly accessible via an AWS Load Balancer.*

Deleting an EKS Cluster

To delete an EKS cluster:

```
eksctl delete cluster --name my-eks-cluster
```

💡 *This removes all associated AWS resources (VPC, worker nodes, etc.).*

Common Issues and Troubleshooting

Problem	Solution
kubectl not working after setup	Run aws eks update-kubeconfig --region <region> --name <cluster-name>

Problem	Solution
Pods stuck in Pending state	Check node availability using kubectl get nodes
Load Balancer Service not working	Ensure type: LoadBalancer is set in the Service YAML
Nodes not joining cluster	Check IAM permissions and security groups

---

Know More (FAQs)

1. What is the difference between EKS and self-managed Kubernetes?

- **EKS** is **fully managed** by AWS, reducing operational overhead.
- **Self-managed Kubernetes** requires users to set up everything manually.

2. How do I manage access to my EKS cluster?

Use **AWS IAM** to grant or restrict access. Example:

```
aws eks update-kubeconfig --region ap-south-1 --name my-cluster
```

3. Can I run EKS without EC2 instances?

Yes! Use **AWS Fargate**, which runs Kubernetes Pods **without managing servers**.

4. How do I monitor my EKS cluster?

Use **Amazon CloudWatch** or **Prometheus** to collect logs and metrics.

5. What is the cost of running an EKS cluster?

AWS charges **$0.10 per hour** for the control plane, plus EC2 and networking costs.

Conclusion

Amazon EKS simplifies running Kubernetes on AWS by **automating cluster management, security, and scaling**. It is a **powerful solution** for deploying containerized applications **without worrying about infrastructure management**.

👉 Try creating an **EKS cluster**, deploying an application, and exposing it using AWS Load Balancer!